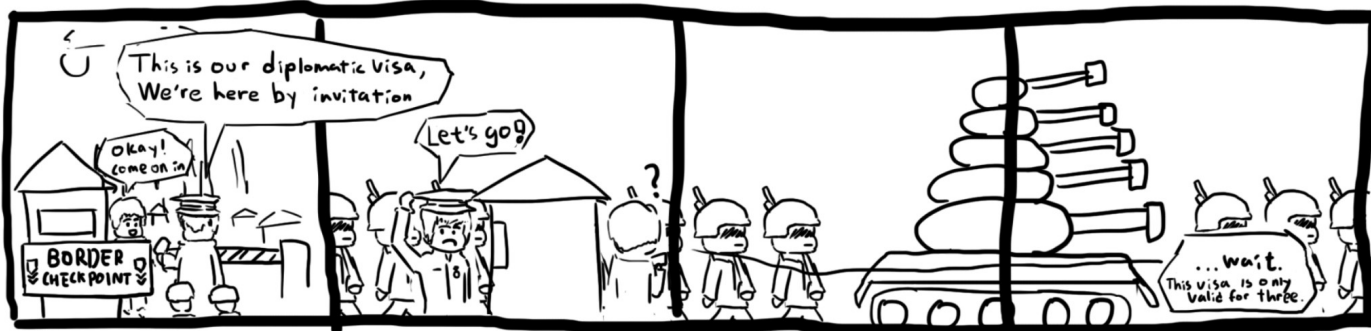


Memory Safety Vulnerabilities

CSE 405 Lecture 14



Last Time: x86 Assembly and Call Stack

CSE 405

- C memory layout
 - **Code** section: Machine code (raw bits) to be executed
 - **Static** section: Static variables
 - **Heap** section: Dynamically allocated memory (e.g. from `malloc`)
 - **Stack** section: Local variables and stack frames
- x86 registers
 - **EBP** register points to the top of the current stack frame
 - **ESP** register points to the bottom of the stack
 - **EIP** register points to the next instruction to be executed
- x86 calling convention
 - When calling a function, the old EIP (RIP) is saved on the stack
 - When calling a function, the old EBP (SFP) is saved on the stack
 - When the function returns, the old EBP and EIP are restored from the stack

Today: Memory Safety Vulnerabilities

CSE 405

- Buffer overflows
 - Stack smashing
 - Memory-safe code
- Integer memory safety vulnerabilities
- Format string vulnerabilities
- Heap vulnerabilities
- Writing robust exploits

Buffer Overflow Vulnerabilities



Textbook Chapter 3.1

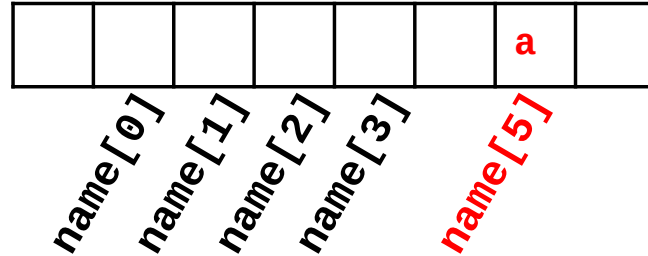
Buffer Overflow Vulnerabilities

CSE 405

- Recall: C has no concept of array length; it just sees a sequence of bytes
- If you allow an attacker to start writing at a location and don't define when they must stop, they can overwrite other parts of memory!

```
char name[4];  
name[5] = 'a';
```

This is technically valid C code,
because C doesn't check bounds!



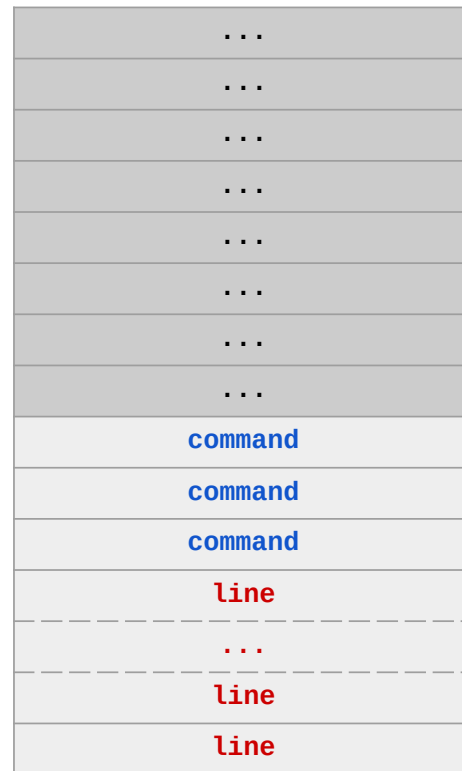
Vulnerable Code

CSE 405

What can go wrong here?

```
char line[512];
char command[] = "/usr/bin/ls";

int main(void) {
    ...
    gets(line);
    ...
    execv(command, ...);
}
```



Vulnerable Code

CSE 405

What can go wrong here?

fnptr is called as a function,
so the EIP jumps to an address
of our choosing!

```
char name[20];  
int (*fnptr)(void);  
  
void vulnerable(void) {  
    ...  
    gets(name);  
    ...  
    fnptr();  
}
```

...
...
...
...
...
...
...
...
...
fnptr
name
name
name
name
name

Top 10 Most Dangerous Software Weaknesses (2023)

CSE 405

Rank	ID	Name	Score
[1]	CWE-787	Out-of-bounds Write	63.72
[2]	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	45.54
[3]	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	34.27
[4]	CWE-416	Use After Free	16.71
[5]	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	15.65
[6]	CWE-20	Improper Input Validation	15.50
[7]	CWE-125	Out-of-bounds Read	14.60
[8]	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')	14.11
[9]	CWE-352	Cross-Site Request Forgery (CSRF)	11.73
[10]	CWE-434	Unrestricted Upload of File with Dangerous Type	10.41



Constructing Exploits

CSE 405

- Place the shellcode *after* the RIP
 - This works because **gets** lets us write as many bytes as we want
 - What should the address be?
- Input: **'A' * 24 + '\x5c\xcd\xff\xbf' + SHELLCODE**
 - 24 bytes of garbage
 - The address of where we placed the shellcode
 - 28 bytes of shellcode

```
void vulnerable(void) {
    char name[20];
    gets(name);
}
```

	'\x00'	...	...	...	
	SHELLCODE				
	SHELLCODE				
	SHELLCODE				
	SHELLCODE				
	SHELLCODE				
	SHELLCODE				
	SHELLCODE				
0xbffffcd5c	SHELLCODE				
0xbffffcd58	'\x5c'	'\xcd'	'\xff'	'\xbf'	RIP
0xbffffcd54	'A'	'A'	'A'	'A'	SFP
0xbffffcd50	'A'	'A'	'A'	'A'	
0xbffffcd4c	'A'	'A'	'A'	'A'	
0xbffffcd48	'A'	'A'	'A'	'A'	
0xbffffcd44	'A'	'A'	'A'	'A'	
0xbffffcd40	'A'	'A'	'A'	'A'	

name

Constructing Exploits

CSE 405

- Input: **SHELLCODE** + 'A' * 12 + '\x40\xcd\xff\xbf' + '\n'
 - 12 bytes of shellcode
 - 12 garbage bytes to overwrite the rest of **name** and the SFP of **vulnerable**
 - The address of where we placed the shellcode

```
void vulnerable(void) {
    char name[20];
    gets(name);
}
```

...	...	...	...	
...	...	...	...	
...	...	...	...	
...	...	...	...	
...	...	...	...	
...	...	...	...	
...	...	...	...	
0xbffcd5c	'\x00'	...	...	
0xbffcd58	'\x40'	'\xcd'	'\xff'	'\xbf'
0xbffcd54	'A'	'A'	'A'	'A'
0xbffcd50	'A'	'A'	'A'	'A'
0xbffcd4c	'A'	'A'	'A'	'A'
0xbffcd48	SHELLCODE			
0xbffcd44	SHELLCODE			
0xbffcd40	SHELLCODE			

RIP

SFP

name

Constructing Exploits

CSE 405

- Alternative: 'A' * 12 + **SHELLCODE** + '\x4c\xcd\xff\xbf' + '\n'
 - The address changed! Why?
 - We placed our shellcode at a different address (**name + 12**)!

```
void vulnerable(void) {
    char name[20];
    gets(name);
}
```

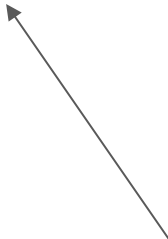
	...	...	...	...	
	...	...	...	...	
	...	...	...	...	
	...	...	...	...	
	...	...	...	...	
	...	...	...	...	
	...	...	...	...	
	...	...	...	...	
0xbfffc5c	'\x00'	...	...	...	
0xbfffc58	'\x4c'	'\xcd'	'\xff'	'\xbf'	RIP
0xbfffc54	SHELLCODE				SFP
0xbfffc50	SHELLCODE				
0xbfffc4c	SHELLCODE				
0xbffcd48	'A'	'A'	'A'	'A'	name
0xbffcd44	'A'	'A'	'A'	'A'	
0xbffcd40	'A'	'A'	'A'	'A'	

Memory-Safe Code

Still Vulnerable Code?

CSE 405

```
void vulnerable?(void) {  
    char *name =  
    malloc(20);  
    ...  
    gets(name);  
    ...  
}
```



Heap overflows are
also vulnerable!

Solution: Specify the Size

```
void safe(void) {  
    char name[20];  
    ...  
    fgets(name, 20,  
    stdin);  
    ...  
}
```

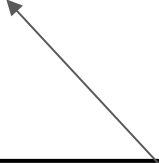
The length parameter specifies the size of the buffer and won't write any more bytes—no more buffer overflows!

Warning: Different functions take slightly different parameters

Solution: Specify the Size

CSE 405

```
void safer(void) {  
    char name[20];  
    ...  
    fgets(name, sizeof(name),  
    stdin);  
    ...  
}
```



sizeof returns the size of the variable (does **not** work for pointers)

Vulnerable C Library Functions

CSE 405

- **gets** - Read a string from stdin
 - Use **fgets** instead
- **strcpy** - Copy a string
 - Use **strncpy** (more compatible, less safe) or **strncpy** (less compatible, more safe) instead
- **strlen** - Get the length of a string
 - Use **strlen** instead (or **memchr** if you really need compatible code)
- ... and more (look up C functions before you use them!)
 - **man** pages are your friend!

Integer Memory Safety Vulnerabilities

Signed/Unsigned Vulnerabilities

CSE 405

Is this safe?

```
void func(int len, char *data) {  
    char buf[64];  
    if (len > 64)  
        return;  
    memcpy(buf, data, len);  
}
```

This is a **signed** comparison, so `len > 64` will be false. But when we call `memcpy()`, casting `-1` to an unsigned type yields `0xffffffff`: another buffer overflow!

`int` is a **signed** type, but `size_t` is an **unsigned** type. What happens if `len == -1`?

```
void *memcpy(void *dest, const void *src, size_t n);
```

Signed/Unsigned Vulnerabilities

CSE 405

Now this is an **unsigned** comparison, and no casting is necessary!

```
void safe(size_t len, char *data) {  
    char buf[64];  
    if (len > 64)  
        return;  
    memcpy(buf, data, len);  
}
```

Integer Overflow Vulnerabilities

CSE 405

Is this safe?

What happens if `len == 0xffffffff`?

```
void func(size_t len, char *data) {  
    char *buf = malloc(len + 2);  
    if (buf == NULL)  
        return;  
    memcpy(buf, data, len);  
    buf[len] = '\n';  
    buf[len + 1] = '\0';  
}
```

`len + 2 == 1`, enabling a heap overflow!

Integer Overflow Vulnerabilities

CSE 405

```
void safe(size_t len, char *data) {  
    if (len > SIZE_MAX - 2)  
        return;  
    char *buf = malloc(len + 2);  
    if (buf == NULL)  
        return;  
    memcpy(buf, data, len);  
    buf[len] = '\n';  
    buf[len + 1] = '\0';  
}
```

It's clunky, but you need to check bounds whenever you add to integers!

Integer Overflow in the Wild

CSE 405



9 to 5 Linux

[Link](#)

New Linux Kernel Vulnerability Patched in All Supported Ubuntu Systems, Update Now

Marius Nestor

January 19, 2022

Discovered by William Liu and Jamie Hill-Daniel, the new security flaw (CVE-2022-0185) is an integer underflow vulnerability found in Linux kernel's file system context functionality, which could allow an attacker to crash the system or run programs as an administrator.

How Does This Vulnerability Work?

CSE 405

- The entire kernel (operating system) patch:
 - `if (len > PAGE_SIZE - 2 - size)`
 - + `if (size + len + 2 > PAGE_SIZE)`
 - `return invalf(fc, "VFS: Legacy: Cumulative options too large)`
- Why was the original code (`len > PAGE_SIZE - 2 - size`) vulnerable?

How Does This Vulnerability Work?

CSE 405

- The entire kernel (operating system) patch:
 - `if (len > PAGE_SIZE - 2 - size)`
 - + `if (size + len + 2 > PAGE_SIZE)`
 - `return invalf(fc, "VFS: Legacy: Cumulative options too large)`
- Why was the original code (`len > PAGE_SIZE - 2 - size`) vulnerable?
 - `PAGE_SIZE`, `size`, and `len` are unsigned

How Does This Vulnerability Work?

CSE 405

- The entire kernel (operating system) patch:
 - `if (len > PAGE_SIZE - 2 - size)`
 - + `if (size + len + 2 > PAGE_SIZE)`
 - `return invalf(fc, "VFS: Legacy: Cumulative options too large)`
- Why was the original code (`len > PAGE_SIZE - 2 - size`) vulnerable?
 - `PAGE_SIZE`, `size`, and `len` are unsigned
 - If `size` is larger than `PAGE_SIZE`, then `PAGE_SIZE - 2 - size` will trigger a negative overflow to `0xFFFFFFFF`
- Result: An attacker can bypass the length check and write data into the kernel

Java's Binary Search Integer Overflow

CSE 405



42



13

A bug was found in Java after almost 9 years of hiding.

#java #computerscience

<https://dev.to/matheusgomes062/a-bug-was-found-in-java-after-almost-9-years-of-hiding-2d4k>

Java's Binary Search Integer Overflow

CSE 405

```
int mid = (low + high) / 2;
```

```
int mid = low + ((high - low) /  
2);
```

Format String Vulnerabilities

printf behavior

CSE 405

- **printf** accepts a variable number of arguments
 - How does it know how many arguments that it received?
 - It infers it from the first argument: the format string!
 - Example: `printf("One %s costs %d", fruit, price)`
 - What happens if the arguments are mismatched?

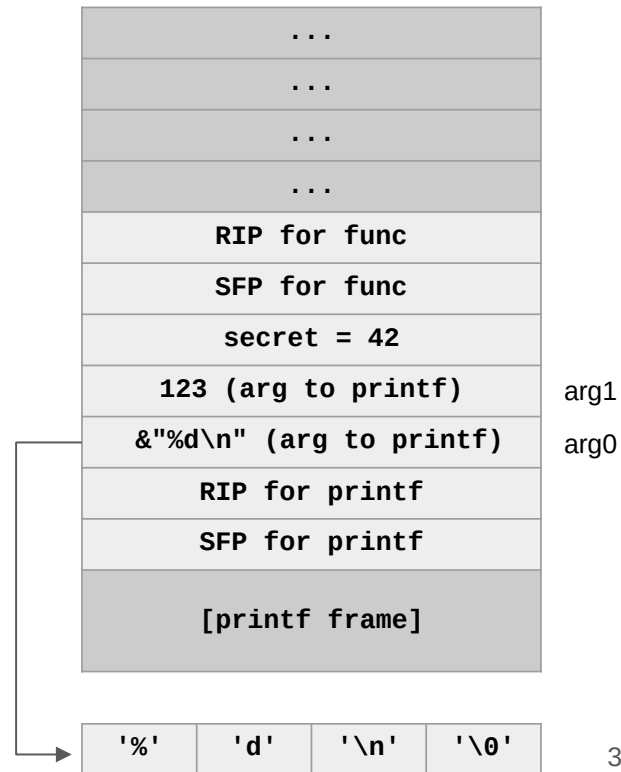
printf behavior

CSE 405

```
void func(void) {  
    int secret = 42;  
    printf("%d\n", 123);  
}
```

printf assumes that there is 1 more argument because there is one format specifier (%d), so it will look 4 bytes up the stack for the argument

What if there is no argument?

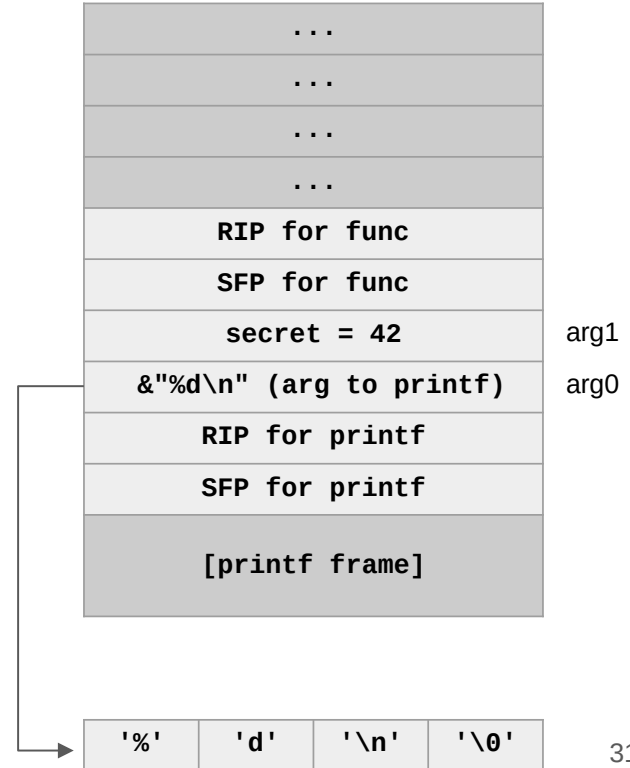


printf behavior

CSE 405

```
void func(void) {  
    int secret = 42;  
    printf("%d\n");  
}
```

Because the format string contains the **%d**, it will still look 4 bytes up -- and print the value of **secret**!



Format String Vulnerability Walkthrough

CSE 405

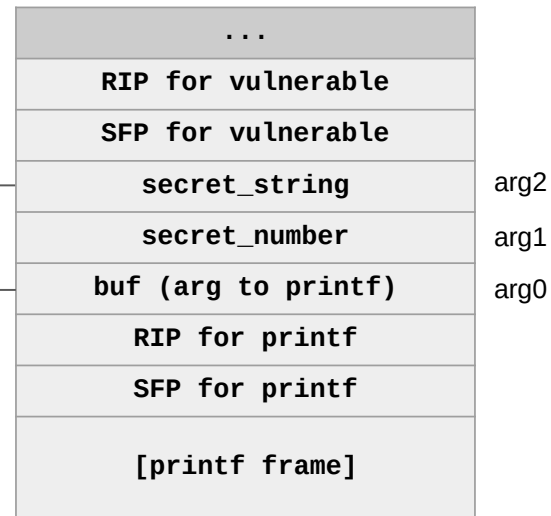
Input: **%d%s**

Output:
42pancake

```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

We're calling **printf("%d%s")**. **printf** reads its first argument (arg0), sees two format specifiers, and expects two more arguments (arg1 and arg2).



'\0'			
'%'	'd'	'%'	's'
'a'	'k'	'e'	'\0'
'p'	'a'	'n'	'c'

Format String Vulnerabilities

CSE 405

- **printf** can also write values using the **%n** specifier
 - **%n** treats the next argument as a **pointer** and writes the number of bytes printed so far to that address (usually used to calculate output spacing)
 - **printf("item %d:%n", 3, &val)**
stores 7 in **val**
 - **printf("item %d:%n", 987, &val)**
stores 9 in **val**
 - **printf("000%n")**
 - **Writes** the value 3 to the memory location **pointed to** by address located 8 bytes above the RIP of **printf**

```
void vulnerable(void) {  
    char buf[64];  
    if (fgets(buf, 64, stdin) == NULL)  
        return;  
    printf(buf);  
}
```

Format String Vulnerability Walkthrough

CSE 405

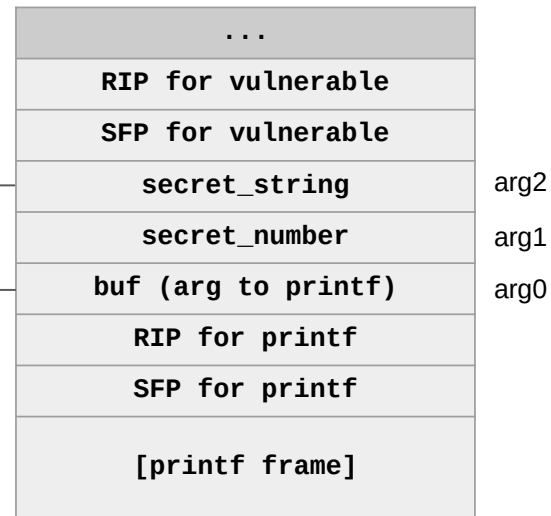
Input: **%d%n**

Output:

```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

We're calling **printf("%d%n")**. **printf** reads its first argument (arg0), sees two format specifiers, and expects two more arguments (arg1 and arg2).



'\0'			
'%'	'd'	'%'	'n'
'a'	'k'	'e'	'\0'
'p'	'a'	'n'	'c'

Format String Vulnerability Walkthrough

CSE 405

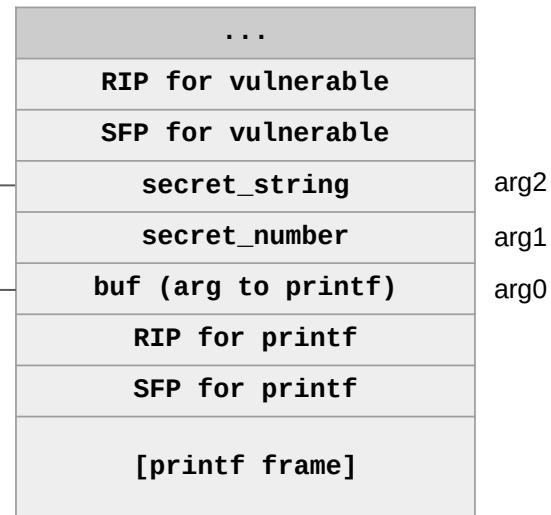
Input: **%d%n**

Output:
42

```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

The first format specifier **%d** says to treat the next argument (arg1) as an integer and print it out.



'\0'			
'%'	'd'	'%'	'n'
'a'	'k'	'e'	'\0'
'p'	'a'	'n'	'c'

Format String Vulnerability Walkthrough

CSE 405

Input: **%d%n**

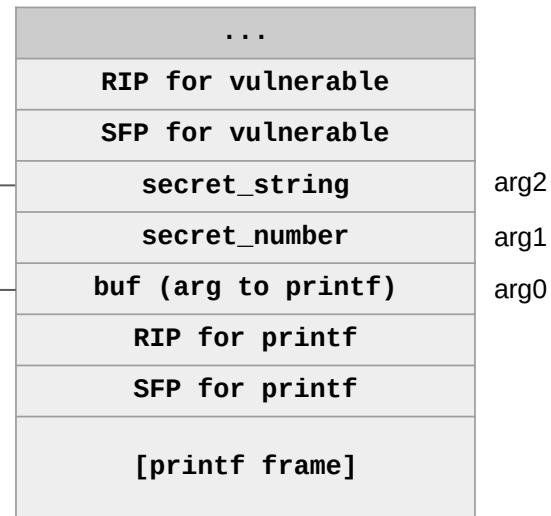
Output:
42

```
char buf[64];

void vulnerable(void) {
    char *secret_string = "pancake";
    int secret_number = 42;
    if (fgets(buf, 64, stdin) == NULL)
        return;
    printf(buf);
}
```

The second format specifier **%n** says to treat the next argument (arg2) as a pointer, and write the number of bytes printed so far to the address at arg2.

We've printed 2 bytes so far, so the number 2 gets written to **secret_string**.



'\0'			
'%'	'd'	'%'	'n'
'a'	'k'	'e'	'\0'
0x02	0x00	0x00	0x00

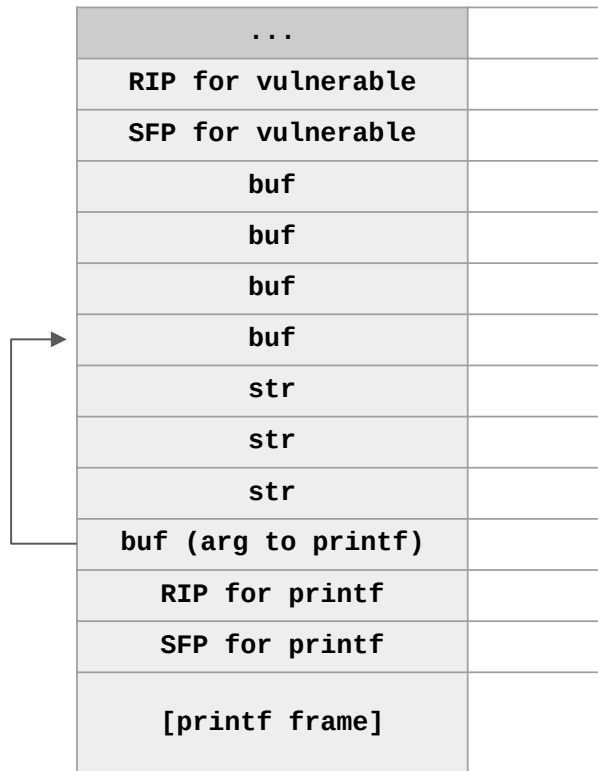
Format Strings: Stack Diagram

CSE 405

```
void vulnerable(void) {  
    char buf[16];  
    char str[12];  
    fgets(buf, 16, stdin);  
    printf(buf);  
}
```

This is the stack diagram while **printf** is being called.

Where does **printf** look for arguments?



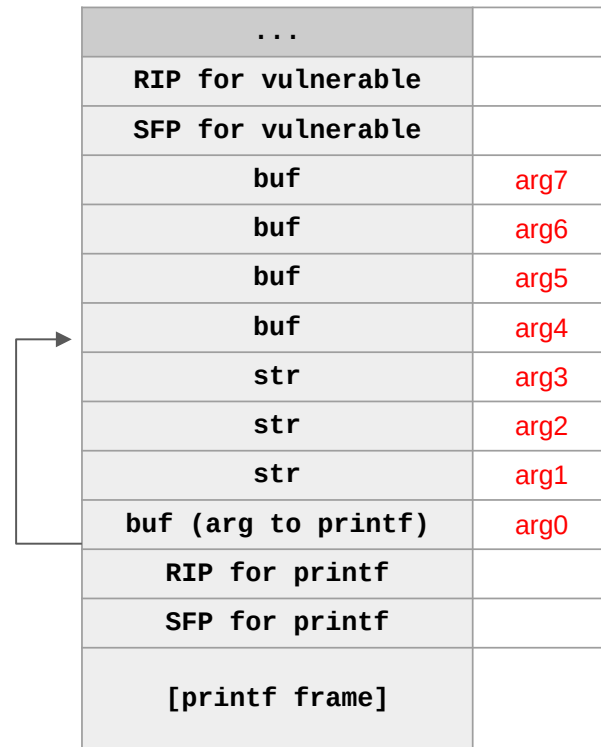
Format Strings: Stack Diagram

CSE 405

```
void vulnerable(void) {  
    char buf[16];  
    char str[12];  
    fgets(buf, 16, stdin);  
    printf(buf);  
}
```

We've labeled which values in memory **printf** will interpret as arguments.

For example, if **buf** has 4 percent formatters, **printf** will match the last percent formatter with arg4.



Write 100 to 0xdeadbeef

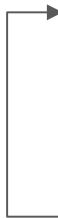
CSE 405

Attack scenario: Write the number 100 to memory address **0xdeadbeef**.

What input should the attacker supply?

```
void vulnerable(void) {  
    char buf[16];  
    char str[12];  
    fgets(buf, 16,  
    stdin);  
    printf(buf);  
}
```

...	
RIP for vulnerable	
SFP for vulnerable	
buf	arg7
buf	arg6
buf	arg5
buf	arg4
str	arg3
str	arg2
str	arg1
buf (arg to printf)	arg0
RIP for printf	
SFP for printf	
[printf frame]	



Write 100 to 0xdeadbeef

CSE 405

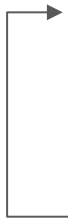
Recall: When **printf** sees a **%n**, it takes the next unused argument, treats it like an address, and writes *the number of bytes printed so far* to that address.

When **printf** sees the **%n**, two things need to be true:

- Control *where* we write: The next unused argument on the stack should be **0xdeadbeef**.
- Control *what* we write: *The number of bytes printed so far* should be 100.

```
void vulnerable(void) {  
    char buf[16];  
    char str[12];  
    fgets(buf, 16,  
    stdin);  
    printf(buf);  
}
```

...	
RIP for vulnerable	
SFP for vulnerable	
buf	arg7
buf	arg6
buf	arg5
buf	arg4
str	arg3
str	arg2
str	arg1
buf (arg to printf)	arg0
RIP for printf	
SFP for printf	
[printf frame]	



Write 100 to 0xdeadbeef

CSE 405

When **printf** sees the **%n**, two things need to be true:

- Control *where* we write: The next unused argument on the stack should be **0xdeadbeef**.
- Control *what* we write: *The number of bytes printed so far* should be 100.

Consider this exploit. What does it look like in memory?

Input:	0xdeadbeef	%94c	%c	%c	%n
--------	------------	------	----	----	----

```
void vulnerable(void) {  
    char buf[16];  
    char str[12];  
    fgets(buf, 16,  
    stdin);  
    printf(buf);  
}
```

...	
RIP for vulnerable	
SFP for vulnerable	
buf	arg7
buf	arg6
buf	arg5
buf	arg4
str	arg3
str	arg2
str	arg1
buf (arg to printf)	arg0
RIP for printf	
SFP for printf	
[printf frame]	

Write 100 to 0xdeadbeef

CSE 405

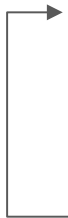
When writing to memory, the percent formatters take up multiple bytes of memory.

For example, **%94c** is 4 characters and takes up 4 bytes of memory.

Input:	0xdeadbeef	%94c	%c	%c	%n
# chars used:	4	4	2	2	2

```
void vulnerable(void) {  
    char buf[16];  
    char str[12];  
    fgets(buf, 16,  
    stdin);  
    printf(buf);  
}
```

...	
RIP for vulnerable	
SFP for vulnerable	
(buf) %n	arg7
(buf) %c%c	arg6
(buf) %94c	arg5
(buf) 0xdeadbeef	arg4
str	arg3
str	arg2
str	arg1
buf (arg to printf)	arg0
RIP for printf	
SFP for printf	
[printf frame]	



Write 100 to 0xdeadbeef

CSE 405

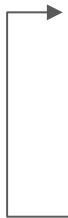
Control *where* we write: The next unused argument on the stack should be **0xdeadbeef**.

- Each percent formatter “uses up” or “consumes” one argument on the stack.
- We added **%c** arguments to “consume” or “skip past” **str**, so that the **%n** argument aligns with **arg4**, where we put **0xdeadbeef**.

Input:	0xdeadbeef	%94c	%c	%c	%n
# chars used:	4	4	2	2	2
Consumes:	N/A	arg1	arg2	arg3	arg4

```
void vulnerable(void) {  
    char buf[16];  
    char str[12];  
    fgets(buf, 16,  
    stdin);  
    printf(buf);  
}
```

...	
RIP for vulnerable	
SFP for vulnerable	
(buf) %n\0	arg7
(buf) %c%c	arg6
(buf) %94c	arg5
(buf) 0xdeadbeef	arg4
str	arg3
str	arg2
str	arg1
buf (arg to printf)	arg0
RIP for printf	
SFP for printf	
[printf frame]	



Write 100 to 0xdeadbeef

CSE 405

Control *what* we write: *The number of bytes printed so far* should be 100.

- **%94c** prints the next argument on the stack as a character, padded to 94 bytes. (Also works if you switch 94 with other numbers.)
- **0xdeadbeef** and the **%c** formatters also caused characters to be printed, so we needed $100 - 4 - 1 - 1 = 94$ padding bytes.

Input:	0xdeadbeef	%94c	%c	%c	%n
# chars used:	4	4	2	2	2
Consumes:	N/A	arg1	arg2	arg3	arg4
# bytes printed:	4	94	1	1	0

```
void vulnerable(void) {
    char buf[16];
    char str[12];
    fgets(buf, 16,
stdin);
    printf(buf);
}
```

...	
RIP for vulnerable	
SFP for vulnerable	
(buf) %n\0	arg7
(buf) %c%c	arg6
(buf) %94c	arg5
(buf) 0xdeadbeef	arg4
str	arg3
str	arg2
str	arg1
buf (arg to printf)	arg0
RIP for printf	
SFP for printf	
[printf frame]	

Write 100 to 0xdeadbeef

CSE 405

How would you modify this exploit to write to address **0xbfff1234** instead of **0xdeadbeef**?

How would you modify this exploit to write 89 to **0xbfff1234** instead of writing 100?

Input:	0xdeadbeef	%94c	%c	%c	%n
# chars used:	4	4	2	2	2
Consumes:	N/A	arg1	arg2	arg3	arg4
# bytes printed:	4	94	1	1	0

```
void vulnerable(void) {  
    char buf[16];  
    char str[12];  
    fgets(buf, 16,  
    stdin);  
    printf(buf);  
}
```

...	
RIP for vulnerable	
SFP for vulnerable	
(buf) %n\0	arg7
(buf) %c%c	arg6
(buf) %94c	arg5
(buf) 0xdeadbeef	arg4
str	arg3
str	arg2
str	arg1
buf (arg to printf)	arg0
RIP for printf	
SFP for printf	
[printf frame]	

Format String Vulnerabilities: Defense

CSE 405

```
void vulnerable(void) {  
    char buf[64];  
    if (fgets(buf, 64, stdin) == NULL)  
        return;  
    printf("%s", buf);  
}
```

Never use untrusted input in the first argument to **printf**.

Now the attacker can't make the number of arguments mismatched!

Summary: Memory Safety Vulnerabilities

CSE 405

- **Buffer overflows:** An attacker overwrites unintended parts of memory
 - **Stack smashing:** An attacker overwrites saved registers on the stack
 - **Memory-safe code:** Fixing code to avoid buffer overflows
- **Integer memory safety vulnerabilities:** An attacker exploits how integers are represented in C memory
- **Format string vulnerabilities:** An attacker exploits the arguments to printf
- **Writing robust exploits:** Making exploits work in different environments by using NOP sleds

Defending Against Memory Safety Vulnerabilities

CSE 405

- What are some approaches to defending against memory safety vulnerabilities?
 - Use safer programming languages.
 - Languages that automatically check bounds and prevent unsafe memory access (e.g., Java, Python, Rust, Go, C#).
 - Learn to write memory-safe code.
 - Use safe libraries: fgets, strncpy, snprintf, etc.
 - Use tools for analyzing and patching insecure code.
 - Penetration testing ⇒ Pay someone to break your system
 - Fuzz testing ⇒ Random inputs
 - Valgrind, code reviews.

Defending Against Memory Safety Vulnerabilities

CSE 405

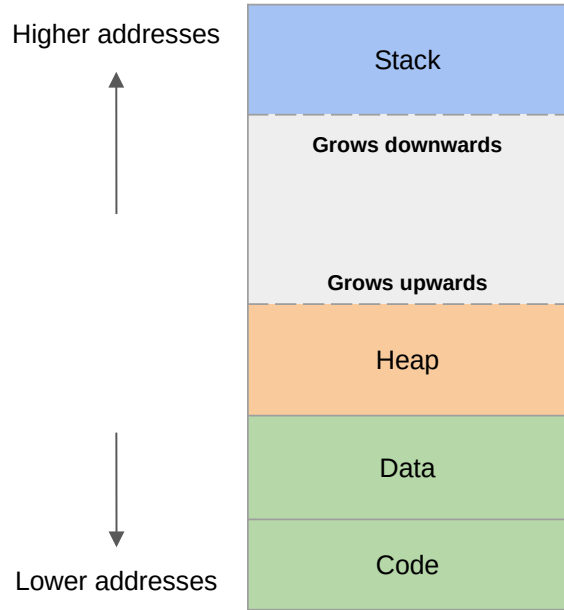
- Add mitigations that make it harder to exploit common vulnerabilities.
 - Non-Executable Pages (DEP/W^X)
 - Also known as: **W^X** (write XOR execute), **DEP** (Data Execution Prevention)
 - Memory is either writable or executable, not both.
 - Page table entries have a writable bit and an executable bit that can be set to achieve this behavior
 - Stack Canaries
 - Special value on stack checked before return.
 - If this value changes, crash the program.
 - ASLR: Address Space Layout Randomization
 - Put each segment of memory in a different, random location each time the program is run
 - ASLR can shuffle all four segments of memory



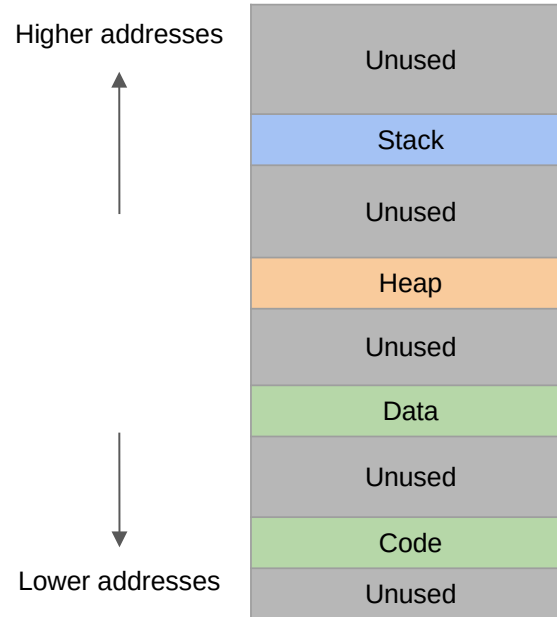
[Read here](#) if you are interested to learn more about canaries

Recall: x86 Memory Layout

CSE 405



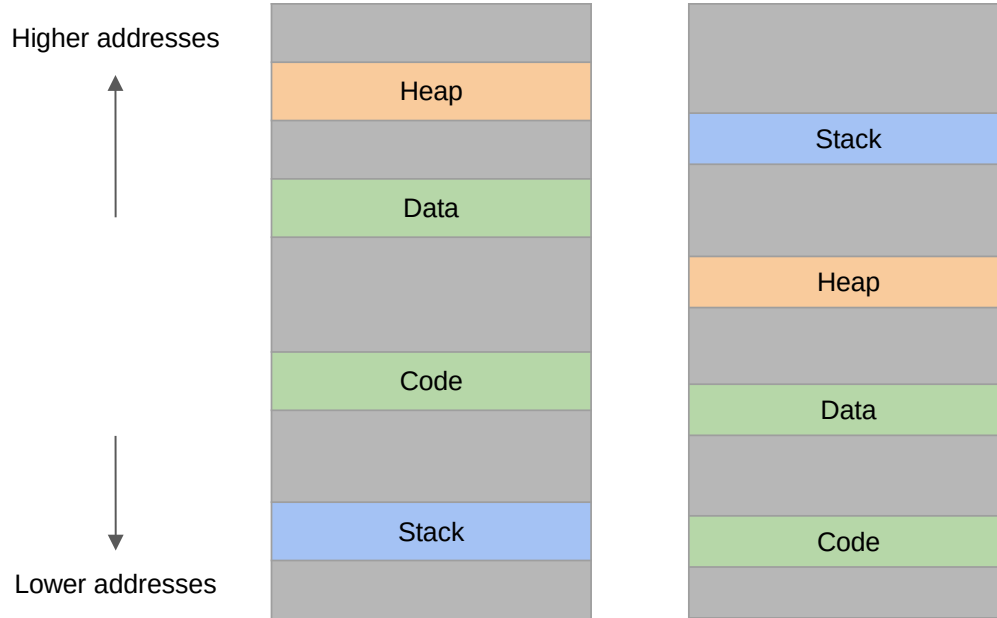
In theory, x86 memory layout looks like this...



...but in practice, it usually looks like this (mostly empty)!

Recall: x86 Memory Layout

CSE 405



Idea: Put each segment of memory in a different location each time the program is run